

FPGA IMPLEMENTATION OF LOW COMPLEXITY CARRIER RECOVERY FOR NEXT GENERATION 16QAM COHERENT OPTICAL INTRA DATACENTER LINKS

SIMON COUDENYS
STUDENT NUMBER: 02101804

Supervisor: prof. dr. ir. Peter Ossieur, ir. Gertjan Coudyzer
Counsellor: Jonas De Busscher, Axl Bomhals, Dr. Jakob Declercq

Master's dissertation submitted in order to obtain the academic degree of Master of Science in Electronics
and ICT Engineering Technology - Embedded Systems

This page is intentionally left blank.

This page is intentionally left blank.

FPGA IMPLEMENTATION OF LOW COMPLEXITY CARRIER RECOVERY FOR NEXT GENERATION 16QAM COHERENT OPTICAL INTRA DATACENTER LINKS

SIMON COUDENYS

STUDENT NUMBER: 02101804

Supervisor: prof. dr. ir. Peter Ossieur, ir. Gertjan Coudyzer

Counsellor: Jonas De Busscher, Axl Bomhals, Dr. Jakob Declercq

Master's dissertation submitted in order to obtain the academic degree of Master of Science in Electronics
and ICT Engineering Technology - Embedded Systems

This page is intentionally left blank.

Acknowledgements

Laboris magna irure quis sit et veniam voluptate duis elit esse do excepteur. Officia laboris adipisicing laboris duis ut cillum aliquip. Do veniam aute non do cillum sit pariatur consectetur adipisicing qui. Lorem Lorem veniam minim exercitation cupidatat tempor do aute dolore. Nostrud adipisicing tempor quis qui fugiat sint quis ut aute eiusmod Lorem enim veniam. Elit labore Lorem do proident amet anim exercitation culpa enim cillum qui. Aliquip reprehenderit commodo labore minim laborum mollit ut veniam in irure. Minim sit in non aliqua ut elit veniam irure pariatur non velit consequat. Cupidatat veniam occaecat culpa eu irure aute consectetur dolore culpa deserunt quis officia do. Aliqua aliqua eu velit excepteur culpa ea dolore occaecat. Ea est cillum nulla ea cupidatat esse adipisicing ut. Cillum amet esse ex ex enim elit excepteur ullamco in qui aliqua Lorem veniam. Sunt consectetur aliquip commodo exercitation pariatur id tempor adipisicing aliqua. Fugiat cillum ea irure laborum consectetur ipsum. Excepteur laboris eu veniam ex ad consequat esse id ullamco ea cupidatat ad nisi. Aute nisi velit excepteur adipisicing dolore pariatur sunt. Dolore mollit cupidatat esse sint ipsum est nulla nisi eu cupidatat excepteur sint cillum. Culpa irure magna culpa eiusmod ut nisi ipsum veniam elit est occaecat veniam. Sunt adipisicing eu minim qui ea laboris ullamco ut. Aliquip occaecat in occaecat ut tempor dolore sunt consequat qui cillum reprehenderit. Voluptate commodo cupidatat ullamco sint aliquip veniam consequat do. Nisi velit excepteur aute laboris aliqua fugiat minim occaecat quis non incididunt. Lorem velit laboris excepteur minim aute laborum ea occaecat duis. Est proident dolor culpa in. Ex sit sint voluptate esse fugiat laborum aliquip laboris quis irure fugiat eiusmod. Dolor occaecat deserunt quis occaecat anim ullamco occaecat dolor. Ea duis eiusmod veniam in fugiat aute dolore officia sint proident deserunt irure est non. Id nulla velit velit aliqua duis dolor Lorem. Sit labore qui dolor occaecat sit irure eiusmod fugiat. In aliqua nisi est in. Tempor aliqua labore magna fugiat incididunt sunt minim sit ut dolor consequat. Cupidatat ex ut nostrud sint officia cillum consequat proident aliquip voluptate. Aliqua reprehenderit dolore eiusmod non. Esse eu exercitation laboris aliqua enim magna sunt id veniam anim. Laboris in nostrud aliqua anim nostrud adipisicing sit velit id eiusmod commodo Lorem deserunt commodo. Eiusmod commodo culpa ad tempor. Excepteur sint fugiat ex veniam esse fugiat laboris mollit. Exercitation id ex culpa aliquip in magna quis consequat. Qui aliquip consequat amet ad laboris non tempor minim eiusmod officia aliquip ullamco. Anim adipisicing magna do et dolore. Quis non aliquip eiusmod cupidatat. Excepteur adipisicing minim cupidatat adipisicing irure. Sunt proident fugiat nulla ea minim esse non. Exercitation dolor non nulla est et adipisicing est dolore pariatur reprehenderit cillum eiusmod. Eu sunt aliquip quis ea nulla ea veniam quis consequat. Veniam proident id eu cillum enim sit enim id sunt. Laboris magna irure quis sit et veniam voluptate duis elit esse do excepteur. Officia laboris adipisicing laboris duis ut cillum aliquip. Do veniam aute non do cillum sit pariatur consectetur adipisicing qui. Lorem Lorem veniam minim exercitation cupidatat tempor do aute dolore. Nostrud adipisicing tempor quis qui fugiat sint quis ut aute eiusmod Lorem enim veniam. Elit labore Lorem do proident amet anim exercitation culpa enim cillum qui. Aliquip reprehenderit commodo labore minim laborum mollit ut veniam in irure. Minim sit in non aliqua ut elit veniam irure pariatur non velit consequat. Cupidatat veniam occaecat culpa eu irure aute consectetur dolore culpa deserunt quis officia do. Aliqua aliqua eu velit excepteur culpa ea dolore occaecat. Ea est cillum nulla ea cupidatat esse adipisicing ut. Cillum amet esse ex ex enim elit excepteur ullamco in qui aliqua Lorem veniam. Sunt consectetur aliquip commodo exercitation pariatur id tempor adipisicing aliqua. Fugiat cillum ea irure laborum consectetur ipsum. Excepteur laboris eu veniam ex ad consequat esse id ullamco ea cupidatat ad nisi. Aute nisi velit excepteur adipisicing dolore pariatur sunt. Dolore mollit cupidatat esse sint ipsum est nulla nisi eu cupidatat excepteur sint cillum. Culpa irure magna culpa eiusmod ut nisi ipsum veniam elit est occaecat veniam. Sunt adipisicing eu minim qui ea laboris ullamco ut. Aliquip occaecat in occaecat ut tempor dolore sunt consequat qui cillum reprehenderit. Voluptate commodo cupidatat ullamco sint aliquip veniam consequat do. Nisi velit excepteur aute laboris aliqua fugiat minim occaecat quis non incididunt. Lorem velit laboris excepteur minim aute laborum ea occaecat duis. Est proident dolor culpa in. Ex sit sint voluptate esse fugiat laborum aliquip laboris quis irure fugiat eiusmod. Dolor occaecat deserunt quis occaecat anim ullamco occaecat dolor. Ea duis eiusmod veniam in fugiat aute dolore officia sint proident deserunt irure est non. Id nulla velit velit aliqua duis dolor Lorem. Sit labore qui dolor occaecat sit irure eiusmod fugiat. In aliqua nisi est in. Tempor aliqua labore magna fugiat

incididunt sunt minim sit ut dolor consequat. Cupidatat ex ut nostrud sint officia cillum consequat proident aliquip voluptate. Aliqua reprehenderit dolore eiusmod non. Esse eu exercitation laboris aliqua enim magna sunt id veniam anim. Laboris in nostrud aliqua anim nostrud adipisicing sit velit id eiusmod commodo Lorem deserunt commodo. Eiusmod commodo culpa ad tempor. Excepteur sint fugiat ex veniam esse fugiat laboris mollit. Exercitation id ex culpa aliquip in magna quis consequat. Qui aliquip consequat amet ad laboris non tempor minim eiusmod officia aliquip ullamco. Anim adipisicing magna do et dolore. Quis non aliquip eiusmod cupidatat. Excepteur adipisicing minim cupidatat adipisicing irure. Sunt proident fugiat nulla ea minim esse non. Exercitation dolor non nulla est et adipisicing est dolore pariatur reprehenderit cillum eiusmod. Eu sunt aliquip quis ea nulla ea veniam quis consequat. Veniam proident id eu cillum enim sit enim id sunt.

Admission to loan

“The author(s) gives (give) permission to make this master’s dissertation available for consultation and to copy parts of this master’s dissertation for personal use. In the case of any other use, the copyright terms have to be respected, in particular with regard to the obligation to state expressly the source when quoting results from this master’s dissertation.”

Toegang tot bruikleen

“De auteur(s) geeft (geven) de toelating dit afstudeerwerk voor consultatie beschikbaar te stellen en delen van het afstudeerwerk te kopiëren voor persoonlijk gebruik. Elk ander gebruik valt onder de beperkingen van het auteursrecht, in het bijzonder met betrekking tot de verplichting de bron uitdrukkelijk te vermelden bij het aanhalen van resultaten uit dit afstudeerwerk.”

This page is intentionally left blank.

Thesis as part of an exam

“This master’s dissertation is part of an exam. Any comments formulated by the assessment committee during the oral presentation of the master’s dissertation are not included in this text.”

Simon Coudenys, Mei 2026

This page is intentionally left blank.

FPGA implementation of low complexity carrier recovery for next generation 16QAM coherent optical intra datacenter links

by
SIMON COUDENYS

Master's dissertation submitted in order to obtain the academic degree of Master of Science in Electronics and ICT Engineering Technology - Embedded Systems

Supervisor: prof. dr. ir. Peter Ossieur, ir. Gertjan Coudyzer
Counsellor: Jonas De Busscher, Axl Bomhals, Dr. Jakob Declercq

Faculty of Engineering and Architecture
Ghent University

Department of Information Technology
Chair: prof. dr. ir. Peter Ossieur

Abstract - Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Nibh sit amet commodo nulla facilisi nullam. Amet venenatis urna cursus eget nunc scelerisque. Mauris cursus mattis molestie a iaculis at. Est sit amet facilisis magna etiam tempor orci eu. Malesuada nunc vel risus commodo viverra maecenas accumsan. Volutpat diam ut venenatis tellus in metus vulputate. Adipiscing tristique risus nec feugiat in. Orci sagittis eu volutpat odio facilisis mauris sit. Posuere ac ut consequat semper. Amet commodo nulla facilisi nullam vehicula ipsum a arcu cursus. Odio euismod lacinia at quis risus. Natoque penatibus et magnis dis parturient. Eleifend donec pretium vulputate sapien nec. Odio pellentesque diam volutpat commodo sed. Tellus molestie nunc non blandit massa enim nec dui nunc. Turpis egestas integer eget aliquet. Consequat id porta nibh venenatis cras sed. Purus in mollis nunc sed id semper risus in hendrerit. Pellentesque eu tincidunt tortor aliquam nulla facilisi cras fermentum. Porttitor eget dolor morbi non. Vivamus at augue eget arcu dictum. Eget felis eget nunc lobortis. Urna nunc id cursus metus aliquam eleifend mi. Est pellentesque elit ullamcorper dignissim cras. A erat nam at lectus.

Keywords - DSP, Baudrate sampling, 16QAM, Intra datacenter, Coherent optical links

This page is intentionally left blank.

Guidelines for the PhD Symposium Papers

Joris Thybaut

Supervisor(s): Eric Laermans, Luc Dupré

Abstract— This article explains how to format your article for the Proceedings of the FEA PhD Symposium using Microsoft Word as a word processor.

Keywords— layout, MS Word, FEA PhD Symposium

I. INTRODUCTION

This sample file shows what the layout of your paper should be. The styles of this file have been created starting from Word's *NORMAL.DOT* template. This sample file should work for all recent MS Office versions. A polemic on the use of MS Word versus LaTeX as a word processor would probably be never ending [1][2].

We recommend a *double column* style as this makes the article easier to read. The column width is 88,5mm. In this sample file you will find examples for the layout of displayed equations, theorems, tables, figures, etc.

The author of this sample document is an advanced MS Word user, but certainly not an expert. Any comments are welcome...

II. GETTING STARTED

A. Creating A New Paper based on a MS Word template file

The proper use of this template requires that one copies it into the template directory (something like this: C:\Documents and Settings\USERNAME\Application Data\Microsoft\Templates) and that, when opening a new document, one creates a new document from this template. This sample document shows what various styles and elements (tables, figures,...) in this PhD Symposium style look like.

B. 'Help, I seem unable to use MS Word template files'

A probably more easy way to generate a new Microsoft Word document following the suggested guide lines of the Symposium Proceedings, if you fail to follow the proper way, is edit this sample file and to rename it into a .doc file... The easiest way, in the most recent MS Word versions, is to double-click on the template file saved on, e.g., the desktop.

III. PHD SYMPOSIUM PAPER STYLES

A. Style Reference

Here's an alphabetical reference for the styles used in the FEA PhD Symposium papers. Some are borrowed from Word's *NORMAL.DOT*; others are specific to the Symposium

style. Note that paragraphs do not require special styling and should be left with the "Normal" style (10pt Times New Roman).

Abstract— Includes all text for your abstract. Formatting is 9-point bold. (The word "abstract" itself should be italic instead of bold)

Authors— Comprises the entire comma-delimited list of authors, starting with the principal author, i.e., the presenting PhD student.

AuthorInfo— This style is meant for author affiliation information. This style should be unique within the paper. It produces a special footnote on the first page. The footnote has a non-printing symbol as its reference marker so as not to interfere with the numbering of regular footnotes later in the paper.¹

Heading 1— Top-level heading. Auto-numbered, so you shouldn't add Roman numerals of your own. Appears centered on the column with smallcaps.

Heading 2— Level-two heading. Auto-numbered, so you shouldn't add capital letters of your own.

Heading 3— Level-three heading. Auto-numbered, so you shouldn't add Arabic numerals of your own. Note that only heading levels 1 through 3 are supported in the Symposium style. (Deeper subsectioning levels are not recommended and probably totally useless in a two pages long paper.)

KeyWords— Optional list of keywords. If you supply no index terms, this style should be removed entirely.

Lemma— Heading (only) for a lemma.

List Number— Regular, numbered, and bulleted lists. Used just as within Word normally. Each entire list receives the list style, and carriage returns within the list area define list items.

References— Reference section (bibliography). A special kind of numbered list. There should be only one *References* style area in your paper. Auto-numbered, so you shouldn't add numbering of your own.

Theorem— Heading (only) for a theorem. Derived from *Heading 3*.

Title— Full paper title. Should be the first item in the paper. Appears in 24-point Times New Roman type centered on the page.

B. Order of Appearance

Here's a list of styles specific to the Symposium layout in order of appearance:

Title

Authors

AuthorInfo (special footnote)

Abstract

IndexTerms

Heading 1

Heading 2

J. Thybaut is with the Chemical Engineering Department, Ghent University (UGent), Gent, Belgium. E-mail: Joris.Thybaut@UGent.be .

¹ This is a regular, numbered footnote.

Heading 3

References

All other styles may be repeated as often as necessary.

IV. MATH

The incorporation of mathematical formulas in the text is probably one of the major issues discussed when comparing MS Word to LaTeX. LaTeX is indeed famous for its equation editing possibilities, while Microsoft's ordinary Equation Editor serves the needs for a limited number of relatively simple equations but becomes cumbersome when used in texts with lots of extensive equations.

In the Symposium Proceedings style the justification of equations should be centred and the numbering should be right-justified. By pressing <TAB> once, one will reach the centered tabulator stop and can enter the equation. After inserting another <TAB> the right-justified position will be reached and the equation number can be inserted. As concerns the use of fonts (bold, italic,...), we refer to the common guidelines for any scientific work [3].

Here is an example of an equation within a theorem:

Theorem 1 (Theorem name) Consider the system

$$\begin{aligned}\dot{x} &= Ax + Bu \\ y &= Cx + Du\end{aligned}\tag{1}$$

If A is stable, then the pair {A, B} is stabilizable. Moreover this holds for any B.

An automatic equation numbering can be obtained by the use of the appropriate field. To insert an equation number one should go to *insert, field* and select 'Seq' in 'Field names'. After pressing the button 'Field codes' one can enter, e.g., 'Equation', for the automatic numbering of the equations. One can also simply execute the macro 'equation numbering'.

V. MARKUP

Markup defines the parts of your paper: the title, headings, lists, and so forth. Each of these elements is given an appropriate name, and certain formatting instructions are associated with that name. In Word, the mechanism for associating an element name with its formatting conventions is called a style.

Style names appear in the pull down style menu along the left hand side of your Word window. In the more recent version you can let appear a style window with extra possibilities on the right hand side of the screen. Typically, you highlight an area of text that you wished to designate with a certain style, then select the appropriate name on the style menu.

VI. TABLES AND FIGURES

Tables and figures are centre-justified, which can be obtained by pressing <TAB> once. Captions for tables should be defined before the table item itself and should be inserted in a proper way, i.e., via 'Insert', 'Reference', 'Caption' and then select 'Table'. Captions to figures are inserted in an analogous way, but should be put after the figure.

Table 1 The Caption comes Before the Table

	title page	odd page	even page
onesided	leftTEXT	leftTEXT	leftTEXT

twosided	leftTEXT	rightTEXT	leftTEXT
----------	----------	-----------	----------

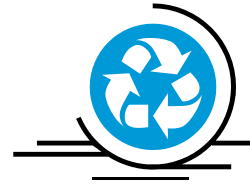


Figure 1 The caption comes after the figure.

VII. REFERENCING

Another LaTeX versus MS Word issue is referencing. Many MS Word users are unaware how to do this or experience problems in doing so. Referencing is only possible when (hidden) bookmarks have been inserted.

A. Hidden bookmarks

When inserting a Table or Figure caption, or when inserting a numbered list (such as the references), hidden bookmarks are created automatically. One can refer to these hidden bookmarks via 'Insert', 'Reference', 'Cross-Reference' and then select the 'reference type', select what should be included (most probably only the number) and then select the actual item you want to refer to. These are references to Figure 1 and Table 1 and to reference [1].

Unexpected things can happen when the hidden bookmark is moved away from its original location. This can, e.g., happen when inserting a reference Y in between two references X and Z. To properly do so one should put the cursor at the end of reference X and press <ENTER>. This is the only way the hidden bookmark of reference Z will stay with that reference and references to this bookmark will refer appropriately. Suppose one hits <ENTER> at the beginning of reference Z the hidden bookmark will not move with the reference and references to this bookmark will point to the newly inserted reference Y instead to reference Z.

When inserting a new item in between two others references are not updated immediately. They are only update when opening the document, printing it or looking at the print preview.

B. User inserted bookmarks

When inserting equation numbers no (hidden) bookmarks are inserted automatically. To be able to refer to an equation number, one has to manually create a bookmark containing that equation number. After selecting the equation number the bookmark can be inserted via 'Insert', 'Bookmark' and then type a name for this bookmark. Once this has been done, a cross-reference to this bookmark can be inserted following the procedure described above. This is a reference to equation (1).

ACKNOWLEDGEMENTS

The authors would like to acknowledge the suggestions of many people.

REFERENCES

- [1] Leslie Lamport, *A Document Preparation System: LaTeX, User's Guide and Reference Manual*, Addison Wesley Publishing Company, 1986.
- [2] Helmut Kopka, *LaTeX, eine einfuhrung*, Addison-Wesley, 1989.

- [3] E. R. Cohen and P. Giacomo, *Symbols, Units, Nomenclature and Fundamental Constants in Physics*, Document I.U.P.A.P.-25 (SUNAMCO 87-1), 1987

This page is intentionally left blank.

Table of Contents

Acknowledgements	vii
Admission to loan	ix
Thesis as part of an exam	xi
Abstract	xiii
Extended Abstract	xv
Table of Contents	xix
List of Figures	xxi
List of Tables	xxi
List of Acronyms	xxiii
1 Introduction	1
1.1 Context and motivation	1
1.2 Problem statement	2
1.3 Objectives of this thesis	2
1.4 Contributions	3
1.5 Thesis outline	4
2 Background	5
2.1 Coherent optical communication systems	5
2.2 Phase noise and frequency offset	6
2.3 Carrier recovery in DSP chains	7
2.4 Blind Phase Search	8
2.5 CORDIC	8
3 Proposed phase recovery architecture	11
3.1 System overview	11
3.2 Serialized blind phase search	11
3.2.1 Adaptation to 16QAM	13
3.2.2 Block size consideration	13
3.3 Spread estimation algorithm	13
3.3.1 Input symbol consideration	14
3.4 Phase unwrapping	14
3.5 Phase compensation	14
3.6 Expected performance	14
4 Hardware implementation	15
4.1 System overview	15
4.2 Data representation and scaling	15
4.3 Serialized blind phase search block	15
4.3.1 Functional operation	16
4.3.2 Data flow	16
4.3.3 Timing flow	18
4.3.4 16QAM-specific adaptations	19
4.4 Spread estimation block	20

4.4.1	Input selector	20
4.4.2	Fourth-power I/Q calculators	22
4.4.3	Spread estimation	22
4.4.4	Sign estimation	22
4.4.5	Filters	23
4.5	Phase correction block	23
4.6	Top-level integration	23
5	Verification and simulation	25
5.1	Simulation model	25
5.2	Test vector generation	25
5.3	VHDL testbench setup	25
5.4	Error analysis	25
5.4.1	Serialized blind phase search	25
5.4.2	Phase spread estimation	25
5.4.3	Phase unwrapper	25
5.4.4	Phase compensator	25
6	Results	27
6.1	Serial blind phase search results	27
6.2	Phase spread estimation results	27
6.3	Error versus SNR	27
6.4	Resource usage	27
6.5	Latency	27
6.6	Comparison with reference method	27
7	Discussion	29
7.1	Limitations	29
7.2	Possible improvements	29
7.3	Design trade-offs	29
8	Conclusion	31
8.1	Summary	31
8.2	Achieved objectives	31
8.3	Future work	31
	Bibliography	33

List of Figures

1.1	This image needs to be referenced still.	1
1.2	Full receiver DSP chain	3
2.1	Near-linear approximation of the total noise process ($\Delta f = 16\text{MHz}$, $\text{dnu} = 0.1\text{MHz}$, $\text{baud} = 32\text{GBaud}$)	6
2.2	FFT-binsize leading to residual phase offset ($F_s = 32\text{GHz}$, $N_{FFT} = 2048$)	7
2.3	Working BPS principle	8
3.1	BPS versus SBPS guided phase estimation	12
3.2	remake figure for more fitting representation	12
4.1	Functional diagram of the full SBPS architecture	15
4.2	Functional diagram of the SBPS stage	16
4.3	Functional diagram of the <i>rotate metric slice</i> unit	16
4.4	Datapath diagram of the full SBPS architecture	17
4.5	Datapath diagram of the SBPS stage	17
4.6	Datapath diagram of the <i>rotate metric slice</i> unit	18
4.7	Timing diagram of the full SBPS algorithm	18
4.8	Timing diagram of the SBPS stage	19
4.9	Timing diagram of the <i>rotate metric slice</i> unit	19
4.10	Top lay-out of SE block	20
4.11	Functional diagram of the SE input selector	20
4.12	Timing flow diagram of the SE input selector	22

List of Tables

This page is intentionally left blank.

List of Acronyms

IMDD	intensity modulation and direct detection
PAM4	pulse amplitude modulation 4-level
QPSK	quadrature phase shift keying
QAM	quadrature amplitude modulation
16QAM	16-state quadrature amplitude modulation
ASIC	application specific integrated circuit
DSP	digital signal processing
BPS	blind phase search
SBPS	serialised binary phase search
SE	spread estimation
PU	phase unwrapper
PC	phase compensator
FPGA	field programmable gate array
CORDIC	coordinate rotation digital computer
IP	intellectual property
VHDL	VHSIC Hardware Description Language
SNR	signal-to-noise ratio
LUT	look-up table
CFO	carrier frequency offset
VHSIC	very high speed integrated circuit
LO	local oscillator
ASE	amplified spontaneous emission
FFT	fast fourier-transform
MUX	multiplexer

This page is intentionally left blank.

Introduction

1.1 Context and motivation

The rapid growth of cloud computing has led to a strong increase in the amount of data exchanged inside modern datacenters. Services such as video and music streaming, cloud storage and online gaming already require large amounts of internal network bandwidth. More recently, the emergence of AI services such as ChatGPT, Gemini and Claude has accelerated this trend even further. These applications rely on large clusters of GPUs and servers that continually exchange data. As a result, the communication links between servers, switches and racks inside a datacenter are becoming an increasingly important bottleneck for these type of services.

To support this growth, datacenter interconnects are evolving towards 800 Gbit/s and 1.6 Tbit/s optical transceivers. These links are expected to play a key role in next-generation AI datacenters, where communication between accelerators and switching equipment must remain both high-speed and energy-efficient [1].

Today, most short-reach intra-datacenter optical links rely on intensity modulation and direct detection (IMDD), often using pulse amplitude modulation 4-level (PAM4) signalling. These techniques are relatively simple and power-efficient, which makes them attractive for short optical links. However, as symbol rates and transmission distances continue to increase, IMDD systems are reaching their limits in terms of chromatic dispersion tolerance, achievable spectral efficiency and receiver sensitivity.

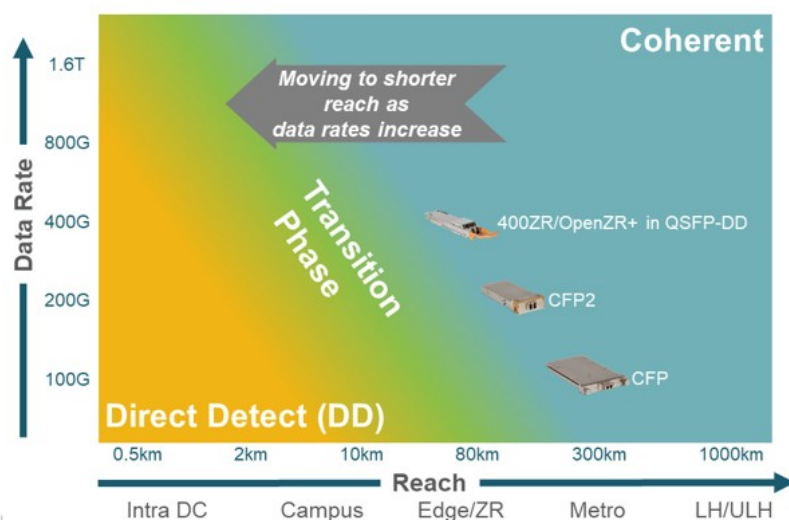


Figure 1.1: This image needs to be referenced still.

Coherent optical communication offers an attractive alternative because it enables the use of higher-order modulation formats such as 16-state quadrature amplitude modulation (16QAM) while providing

improved sensitivity and better tolerance to fiber impairments. Experimental comparisons between coherent and IMDD transceivers for intra-datacenter links have shown that coherent solutions can provide a larger optical power budget while maintaining comparable application specific integrated circuit (ASIC) power consumption [2].

1.2 Problem statement

Although coherent optical communication provides important advantages in terms of spectral efficiency and receiver sensitivity, these benefits come at the cost of increased receiver complexity. In particular, the digital signal processing (DSP) chain of a coherent receiver contains computationally expensive blocks for frequency recovery and carrier phase recovery.

Carrier recovery is responsible for compensating the residual frequency offset and phase noise caused by the finite linewidth of the transmitter and local oscillator lasers. In conventional coherent receivers, this is often achieved through a combination of fourth-power frequency estimation and blind phase search (BPS). The frequency recovery stage performs a coarse estimation of the carrier frequency offset, after which the phase recovery stage compensates the remaining phase rotation and phase noise.

However, both of these techniques become increasingly expensive as baudrates increase. Fourth-power frequency estimation typically relies on large FFT sizes in order to obtain sufficient frequency resolution. At the same time, blind phase search requires many test angles to achieve accurate phase estimation. This results in a significant amount of complex multiplications and memory usage, making carrier recovery one of the dominant contributors to the overall DSP complexity of a coherent receiver.

source if applicable

These complexity challenges are particularly important in intra-datacenter links, where power consumption and latency are critical constraints. In such environments, it is not sufficient for a carrier recovery algorithm to be accurate alone; it must also be implementable efficiently in hardware.

This thesis therefore investigates a low-complexity field programmable gate array (FPGA)-based carrier recovery architecture for baudrate-sampled 16QAM coherent intra-datacenter links. The proposed approach aims to reduce the computational complexity of blind phase search while extending its effective frequency tracking range. This is achieved by combining a serialized blind phase search algorithm with an additional phase spread estimation block, allowing more robust carrier recovery with lower overall hardware cost.

1.3 Objectives of this thesis

A mathematical model of the proposed low-complexity carrier phase recovery system has already been developed in Simulink and will be discussed in more detail in Chapter 3. In this thesis, the implementation of this phase recovery architecture on an FPGA will be presented in detail.

The proposed architecture forms the final stage of the receiver DSP chain, directly before the symbol decision stage.

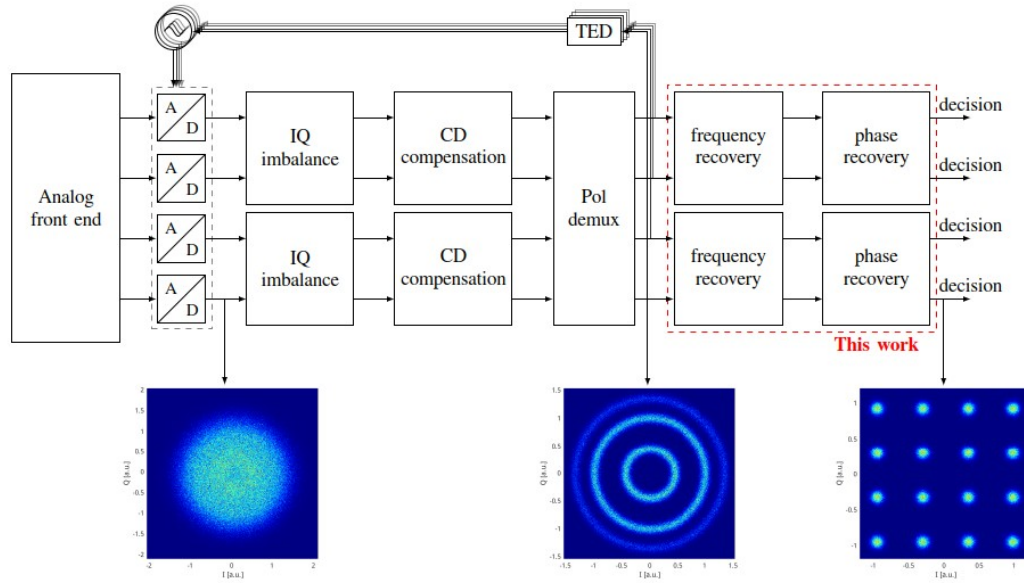


Figure 1.2: Full receiver DSP chain

Because of this position in the processing chain, its performance has a direct impact on the overall receiver quality. The implementation focuses not only on functional correctness, but also on efficient hardware mapping, pipelining and memory usage.

Where multiple implementation strategies are possible, the motivation behind the chosen design decisions will be discussed. Alternative approaches will also be considered, together with the reasons why they were not selected in the final design.

Finally, the performance of the implemented architecture will be evaluated. The proposed carrier recovery system contains several configurable parameters, such as the number of blind phase search stages, block size and phase-correction segments. Increasing these parameters can improve phase estimation accuracy and cycle slip tolerance, but also increases the required hardware resources, latency and power consumption.

The objective of this evaluation is therefore not only to determine the absolute performance of the design, but also to investigate the trade-off between hardware cost and receiver performance under different operating conditions. These conditions include different signal-to-noise ratios, residual frequency offsets and laser linewidths, as these are the dominant impairments affecting coherent carrier recovery.

1.4 Contributions

This thesis presents the design and implementation of a low-complexity phase spread estimation architecture for coherent communication systems.

The main contributions of this work are:

- A fixed-point hardware implementation of the spread estimation algorithm suitable for FPGA deployment.
- The design of a pipelined architecture for phase estimation, including dot-product, reciprocal multiplication, and arccos lookup blocks.
- The development of filtering techniques to improve estimation stability, including moving average filtering and sign majority voting.
- A verification framework using Python reference models and VHDL testbenches to validate the fixed-point implementation.
- A performance evaluation of the proposed architecture in terms of numerical accuracy, latency, and resource usage.

keep this for
after main
body has
been written

1.5 Thesis outline

The remainder of this thesis is structured as follows:

Chapter 2 introduces the theoretical background required for this work. First, coherent optical communication systems are discussed, with a focus on how phase noise and residual frequency offset arise in these systems and how they affect the received signal. This is necessary to understand why carrier recovery is required and what challenges it introduces. Next, existing carrier recovery techniques are reviewed, together with their limitations and motivation for the proposed approach. The digital representation of 16QAM signals is then discussed, since the proposed architecture strongly depends on fixed-point symbol scaling, numerical precision and signal-to-noise ratio (SNR)-related considerations. Finally, a section is dedicated to the coordinate rotation digital computer (CORDIC) algorithm. Unlike the other algorithmic blocks in this thesis, which are developed specifically for this work, the CORDIC is based on an existing intellectual property (IP) core developed by [INSERT REF].

insert ref
CORDIC IP

Chapter 3 presents the proposed phase recovery algorithm from a theoretical perspective. Building on the concepts introduced in the previous chapter, the complete architecture is divided into its main functional blocks, including the serialised binary phase search (SBPS), spread estimation (SE), phase unwrapper (PU) and phase compensator (PC) modules. For each subblock, the operating principle, mathematical model and expected contribution to the overall system are discussed. The theoretical performance of the proposed method is also analysed, based on the work of J. De Busscher in [INSERT REF].

insert ref PA-
PER JONAS

Chapter 4 describes how the theoretical carrier recovery system is translated into an FPGA implementation. This chapter discusses the practical challenges associated with such an implementation, including fixed-point numerical representations, timing closure, and hierarchical design. In addition, it explains how the individual subblocks are combined into a complete processing chain and how dataflow, valid signals and latency alignment are handled throughout the design.

Chapter 5 explains the simulation and verification framework used for the implemented design. The testbenches of the different subblocks are discussed together with their corresponding results. The purpose of this chapter is to show that the hardware implementation behaves in accordance with the theoretical model and to explain any deviations between both. The focus is therefore on functional correctness rather than on overall system performance under different operating conditions. Comparisons between the FPGA implementation and the Python or Simulink reference models are also included to demonstrate that the implemented fixed-point design accurately reproduces the expected behaviour.

Chapter 6 evaluates the overall performance of the implemented carrier recovery system. Separate sections discuss the performance of the SBPS and SE blocks and identify the conditions under which their performance begins to degrade. The influence of different system parameters, such as block size, number of search stages and datapath-bitlengths, is also investigated. In addition, this chapter discusses the hardware resource usage and latency of the final design. The relation between implementation complexity and system performance is analysed in order to determine which parameters offer the most favourable trade-off.

Chapter 7 discusses the limitations of the current implementation and identifies possible improvements for future work. This includes both algorithmic improvements and hardware-related optimizations. Potential extensions such as more advanced look-up table (LUT) implementations, adaptive parameter selection, improved spread estimation for low SNR conditions and support for higher-order modulation formats are considered. In addition, the discussion reflects on the assumptions made throughout the work, such as the assumption that the residual frequency offset changes only slowly between neighbouring blocks. The chapter also provides a broader overview of the main design trade-offs and explains how these trade-offs influenced the final architecture.

find good al-
ternative so-
lutions, these
are purely
speculative

Finally, Chapter 8 summarizes the most important aspects of the work. It revisits the original objectives of the thesis, discusses the extent to which these objectives were achieved and reflects on the main findings of the implementation and evaluation. The final chapter also outlines possible directions for future work, including integration into a complete coherent receiver chain and further optimization of the architecture for practical deployment.

Background

This chapter summarises the background required to contextualise the proposed coherent receiver carrier-recovery architecture. It focuses on (i) short-reach IM/DD links versus coherent detection, (ii) phase-related impairments (carrier frequency offset and laser phase noise), and (iii) representative DSP-based carrier recovery methods used in coherent receivers

2.1 Coherent optical communication systems

Traditional short-reach optical interconnects in data centres have often favoured intensity-modulation/direct-detection (IM/DD) transceivers because they enable simple and power-efficient receiver front-ends. In IM/DD systems, information is conveyed by modulating the received optical intensity (or optical power). At the receiver, a photodiode converts incident optical power into an electrical photocurrent (square-law photodetection), so the receiver directly observes intensity but does not directly access the optical carrier phase [3, 4].

IM/DD links are attractive for short distances and moderate data rates; however, as symbol rates increase, fibre chromatic dispersion becomes increasingly detrimental because group delay varies with optical wavelength, causing waveform distortion and frequency-selective fading for intensity-modulated signals. In addition, the achievable spectral efficiency (information rate per unit bandwidth, in b/s/Hz) is fundamentally constrained when only intensity is detected, because direct detection effectively exploits fewer degrees of freedom than coherent detection [4].

Coherent optical communication systems overcome these limitations by detecting both the amplitude and phase of the optical field and enabling advanced modulation formats such as QPSK and square M-QAM. Coherent detection also supports digital compensation of linear channel impairments (e.g., chromatic dispersion and polarization effects) in the electrical domain [4, 5, 6]. Recent work on data-centre optics evaluates when coherent detection becomes favourable relative to IM/DD as per-lane rates scale, highlighting the trade-off between coherent DSP power/complexity and improved spectral efficiency and impairment tolerance [7].

A typical intradyne coherent receiver combines the received optical field with a local oscillator (LO) laser in a 90° optical hybrid. The hybrid generates orthogonal interference terms that are detected by balanced photodiode pairs, yielding electrical in-phase (I) and quadrature (Q) components. Balanced detection improves sensitivity to the desired beating term and suppresses common-mode intensity noise [5, 8].

After analog-to-digital conversion, the coherent receiver typically performs several DSP operations, including chromatic dispersion compensation, timing recovery, adaptive equalization, carrier frequency recovery and carrier phase recovery [5, 6]. In many DSP chains, dispersion compensation and polarization demultiplexing/equalization are performed prior to carrier phase estimation because commonly used blind equalizers can converge without an explicit carrier-phase reference; in practice, frequency-offset estimation may be placed either before or after equalization depending on acquisition range and implementation constraints [5, 6].

Historically, optical coherent receivers often relied on analogue phase-locked loops (PLLs) for carrier synchronisation. In modern DSP-based coherent receivers, carrier recovery is commonly implemented digitally using feed-forward and/or decision-directed algorithms, which avoids tight analogue loop dynamics and integrates naturally with polarization-diverse digital processing [5, 9, 10].

2.2 Phase noise and frequency offset

The main phase-related impairments in coherent optical communication systems are carrier frequency offset (CFO) and phase noise. Both arise because the transmitter and local oscillator lasers are independent devices and therefore do not share an identical optical frequency or phase reference [8].

Carrier frequency offset occurs because the instantaneous optical frequency of the transmitter laser differs from that of the LO laser. After coherent detection and digitisation, CFO appears as an effective baseband frequency offset and produces a deterministic phase ramp across samples [8]. Mathematically, this phase rotation can be expressed as

$$y[k] = s[k]e^{j(\theta[k] + 2\pi\Delta f T_s k)} + \eta[k] \quad (2.1)$$

where $s[k]$ is the transmitted symbol, $\theta[k]$ is the phase noise, Δf is the carrier frequency offset, T_s is the symbol period and $\eta[k]$ is additive noise [8, 11].

The term $2\pi\Delta f T_s k$ corresponds to a linear phase ramp. A larger frequency offset or a larger symbol duration leads to faster constellation rotation. This is particularly relevant for block-based carrier recovery systems, because the total accumulated phase change within one block influences the probability of phase ambiguities and cycle slips [10].

Besides frequency offset, coherent optical systems are also affected by phase noise. Laser phase noise is commonly modelled as a discrete-time Wiener process (random walk) [11]:

$$\theta[k] = \theta[k-1] + \Delta\theta[k] \quad (2.2)$$

where $\Delta\theta[k]$ is a zero-mean Gaussian increment whose variance is proportional to the combined (beat) linewidth and the sampling interval [11, 10].

For the systems considered in this thesis, the residual CFO after coarse frequency recovery is especially important. While a separate frequency recovery stage can compensate most of the CFO, a small leftover frequency error remains due to the finite resolution of practical estimators [12, 13]. This residual offset causes a nearly linear phase drift across each DSP block.

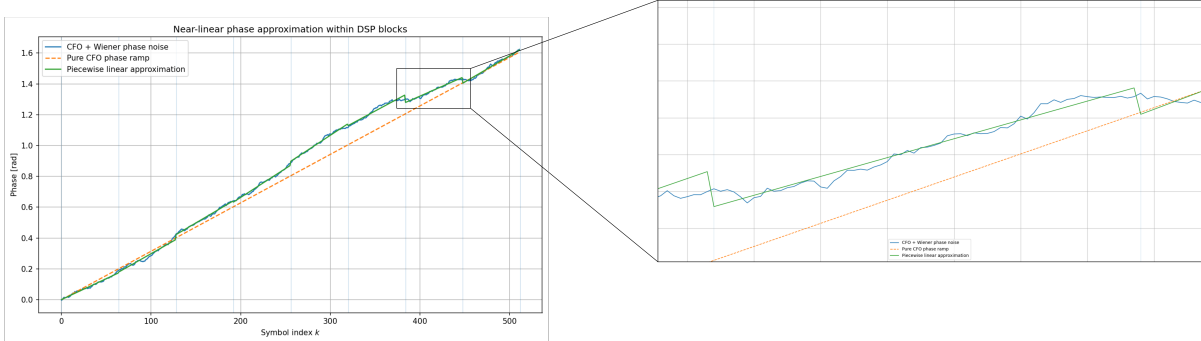


Figure 2.1: Near-linear approximation of the total noise process ($\Delta f = 16\text{MHz}$, $\text{dnu} = 0.1\text{MHz}$, $\text{baud} = 32\text{GBaud}$)

The phase recovery system must therefore not only compensate random phase noise caused by laser linewidth, but also account for remaining within-block phase slope. This motivates the channel model used throughout this thesis, in which the received symbols are impaired by:

- additive white Gaussian noise,
- a residual carrier frequency offset,
- Wiener phase noise caused by laser linewidth.

A coarse FFT-based frequency recovery algorithm is often used before phase recovery. FFT-based CFO estimation provides wide acquisition range at modest complexity [12, 13]. For M-PSK constellations, an M-th power nonlinearity can suppress the data modulation and expose a strong carrier-related spectral line that enables peak-based CFO estimation; related techniques are also adapted for square QAM via partitioning, pilots, or data-aided processing [14].

The estimated frequency offset can then be compensated digitally. However, the accuracy of this estimate depends on the FFT size. A finite FFT resolution leads to a remaining frequency estimation error. Since the FFT bin spacing equals F_s/N_{FFT} , a simple bin-centred peak estimate implies an error on the order of half a frequency bin:

$$\Delta f_e \leq \frac{F_s}{2N_{\text{FFT}}} \quad (2.3)$$

[15, 16].

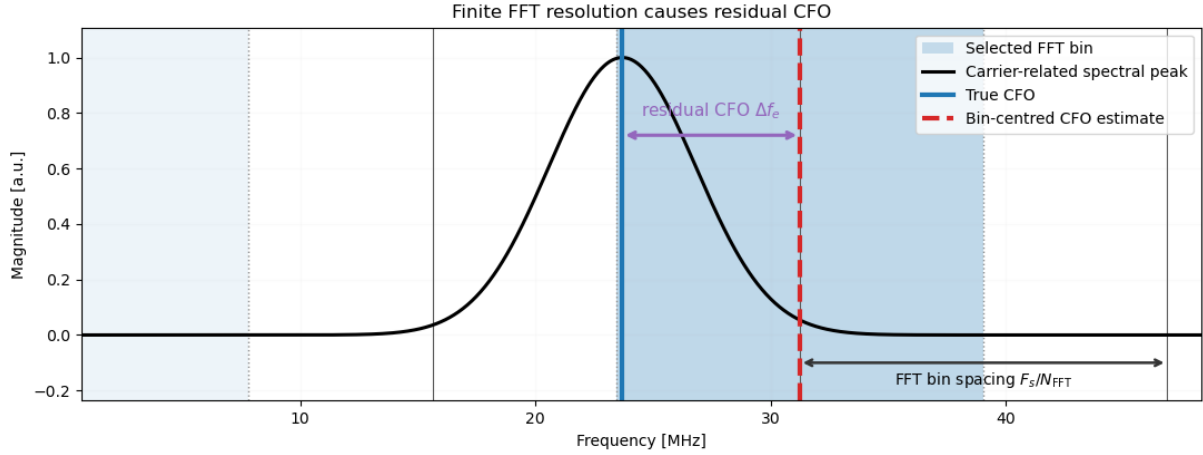


Figure 2.2: FFT-binsize leading to residual phase offset ($F_s = 32\text{GHz}$, $N_{\text{FFT}} = 2048$)

This remaining frequency error is then passed on to the phase recovery stage. If the residual offset is too large, the phase recovery block may no longer be able to track the constellation rotation correctly, which increases the probability of cycle slips [10, 6].

2.3 Carrier recovery in DSP chains

Carrier recovery is the DSP stage responsible for compensating the phase rotations introduced by residual CFO and phase noise. Since these impairments have different characteristics, coherent receivers commonly separate carrier recovery into (i) frequency offset estimation/compensation and (ii) carrier phase estimation [5, 6].

Frequency recovery estimates a single CFO parameter over an observation interval and therefore benefits from aggregating many samples to improve estimator variance; it is not typically meaningful to produce an independent CFO estimate per symbol. By contrast, carrier phase recovery must track a time-varying phase process (Wiener phase noise plus residual CFO), motivating symbol-rate or block-rate phase estimation [12, 10].

A major challenge is that modern coherent DSP systems process many symbols in parallel in order to reduce clock speed requirements. As a result, carrier phase is often estimated only once per processing block instead of once per symbol [6]. Although this significantly reduces computational complexity, it also increases sensitivity to phase drift within the block. If the total phase change over a block becomes too large, the phase recovery algorithm may unwrap to an incorrect $\pi/2$ -shifted hypothesis (cycle slip) for square QAM constellations [10].

If a block contains B symbols and the residual frequency offset is Δf_e , the total phase drift across the block can be approximated by

$$\Delta\phi_{\text{block}} \approx 2\pi\Delta f_e B T_s \quad (2.4)$$

To reduce the probability of cycle slips, a practical design rule is to keep the intra-block phase drift well below the midpoint between neighbouring $\pi/2$ -ambiguous hypotheses, often expressed as $|\Delta\phi_{\text{block}}| \lesssim \pi/4$ [10].

2.4 Blind Phase Search

Blind Phase Search (BPS) is a widely used feed-forward carrier phase estimation method for coherent square M-QAM. Its advantages include pilot-free operation, modulation-format compatibility, and strong tolerance to laser phase noise when sufficient test phases and averaging are used [10, 17].

The principle of BPS is straightforward. A block of received symbols is rotated over a set of candidate test angles. For each candidate angle, the rotated symbols are sliced to the nearest ideal constellation points and an error metric is computed. The candidate angle that minimises the total error is selected as the estimated carrier phase [10].

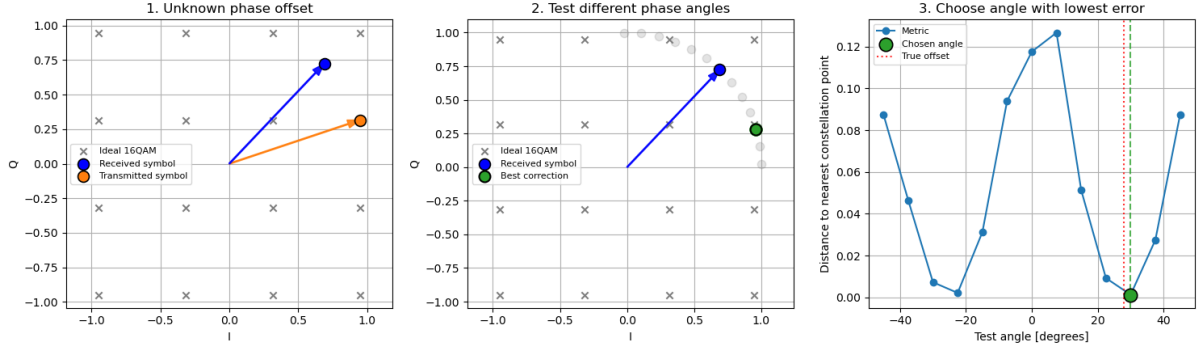


Figure 2.3: Working BPS principle

For a received block $z[k]$, the BPS metric for a test phase ϕ can be written as

$$e(\phi) = \sum_{k=0}^{B-1} |D(z[k]e^{-j\phi}) - z[k]e^{-j\phi}|^2 \quad (2.5)$$

where $D(\cdot)$ denotes the decision operation that maps a rotated symbol to its nearest constellation point. The phase estimate is then given by the test angle that minimises $e(\phi)$.

For square QAM constellations, the objective function in BPS is periodic with period $\pi/2$ due to constellation rotational symmetry. Therefore, it suffices to search any interval of length $\pi/2$; a common centred choice is $[-\pi/4, \pi/4]$ [10]. However, a large number of test phases is usually required to obtain sufficient estimation accuracy, making BPS computationally expensive in high-throughput FPGA implementations [10, 17].

Another limitation of blockwise BPS is that it estimates one average phase per block. When residual CFO induces significant intra-block phase drift, a single blockwise estimate can be insufficient and may increase cycle-slip probability [10]. This thesis therefore extends the traditional blockwise BPS approach by also estimating the phase spread across the block, as explained in Chapter 3.

2.5 CORDIC

The Coordinate Rotation Digital Computer (CORDIC) algorithm implements vector rotations and elementary functions using iterative shift-add operations. Its attractiveness in FPGA/ASIC implementations stems from replacing general multipliers with pipelinable add/subtract and bit-shift datapaths [18, 19].

In the rotation mode of CORDIC, a vector is iteratively rotated by a sequence of predefined micro-rotations. The iterative equations can be written as [18, 19]:

$$x_{i+1} = x_i - d_i y_i 2^{-i} \quad (2.6)$$

$$y_{i+1} = y_i + d_i x_i 2^{-i} \quad (2.7)$$

$$z_{i+1} = z_i - d_i \arctan(2^{-i}) \quad (2.8)$$

where $d_i \in \{-1, +1\}$ determines the direction of rotation at iteration i . After a sufficient number of iterations, the vector is rotated by the desired angle with good precision [19].

In this thesis, CORDIC is used to rotate received symbols by the estimated carrier phase. The implementation leverages an FPGA IP core; design effort therefore focuses on selecting fixed-point formats, pipeline depth/latency, and interface timing to meet throughput and numerical-accuracy requirements [20].

This page is intentionally left blank.

Proposed phase recovery architecture

Here: explain algorithm clearly use diagrams use equations refer to paper but explain your version This chapter = theory of your solution.

3.1 System overview

Assumption that

$$\sum_{i=0}^{B-1} \phi_i \ll 2\pi\Delta f T_s B \quad (3.1)$$

3.2 Serialized blind phase search

The purpose of SBPS is to estimate the carrier phase of the middle of a processing block with much lower complexity than conventional blind phase search. This estimate should be accurate enough to serve as the central anchor point of the proposed phase recovery architecture, and must be in range of the largest CFO error that is possible for a certain frequency recovery architecture. However, because only one phase estimate is produced per block and because the BPS metric is inherently $\pi/2$ -ambiguous for square QAM constellations, additional processing is required to (i) track phase continuity between successive blocks and (ii) reconstruct the phase evolution inside the block. These two tasks are handled by the phase unwrapping and spread estimation blocks, respectively.

In conventional BPS, many candidate phases are evaluated in parallel over the full search interval (as mentioned in section 2.4). Although this leads to robust phase estimation, it also results in a substantial computational cost.

For square quadrature amplitude modulation (QAM) constellations, the search interval is limited to

$$\left[-\frac{\pi}{4}, \frac{\pi}{4}\right),$$

The key idea of SBPS is to approximate the optimal test phase through successive refinement. Instead of testing many candidate phases simultaneously, only two candidate rotations are evaluated at each stage: one on either side of the current estimate. The candidate producing the lower decision-directed metric is retained, after which the search interval is halved and a more refined correction is tested in the next stage.

In the binary variant adopted here, the first stage tests the two hypotheses

$$\phi_1 \in \left\{-\frac{\pi}{8}, +\frac{\pi}{8}\right\},$$

and stage m refines the estimate by adding or subtracting a correction term of magnitude $\pi/2^{m+2}$. Hence, the estimate after stage m can be written recursively as

$$\hat{\phi}_m = \hat{\phi}_{m-1} \pm \frac{\pi}{2^{m+2}}.$$

explain what the purpose is of the 2 stage design, e.g; why SBPS needs to track middle phase, SE the spread

needs to be more detailed

maybe explain why?

maybe here delve into the specific costs

remake figure for more fitting representation

motivate why binary search is chosen?

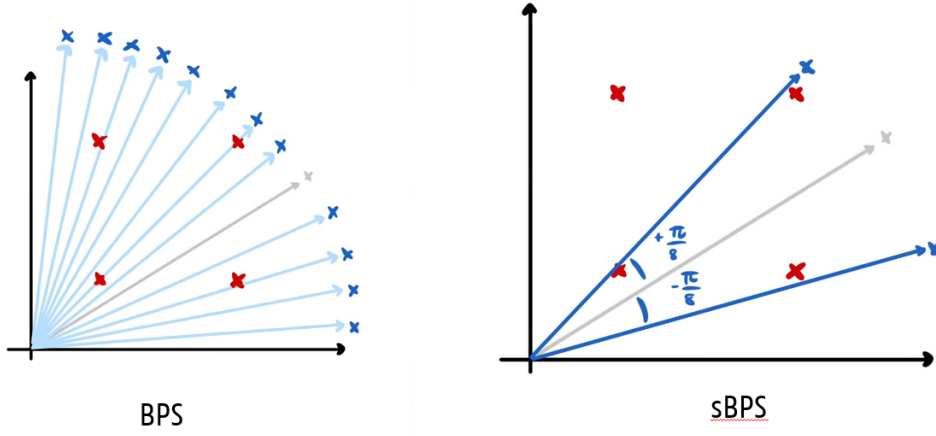


Figure 3.1: BPS versus SBPS guided phase estimation

This staged procedure is equivalent to a binary search over the phase interval and yields an exponentially decreasing phase quantization error as the number of stages increases. The referenced work formulates this refinement explicitly and shows that the worst-case estimation error decreases with each added stage, while the number of candidate rotations only grows linearly with the number of stages [21].

For each candidate phase ϕ , a decision-directed metric is computed over a block of received symbols:

$$J(\phi) = \sum_{k \in \mathcal{W}} |D(r_k e^{-j\phi}) - r_k e^{-j\phi}|^2,$$

where r_k denotes the received symbol, $D(\cdot)$ the slicer, and $\mathcal{W}$ the symbol window used for metric accumulation.

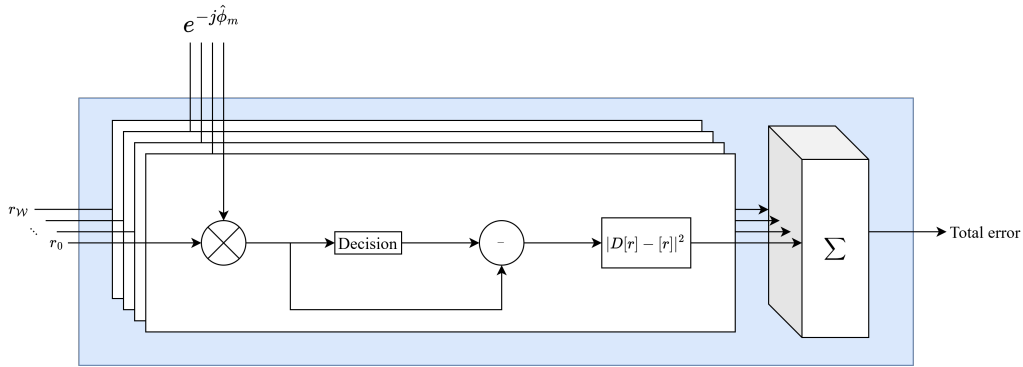


Figure 3.2: remake figure for more fitting representation

The candidate phase yielding the smallest metric is assumed to correspond most closely to the true carrier phase. In this work, the resulting estimate is interpreted as the carrier phase at the middle of the block, denoted by $\hat{\phi}_e$. This interpretation is important, since in the presence of residual frequency offset the carrier phase is not constant over the entire block but varies approximately linearly with symbol index .

maybe useful to explain how complexity scales with accuracy

remake figure for more fitting representation

and laser phase drift is also present! (job of SE)

3.2.1 Adaptation to 16QAM

3.2.2 Block size consideration

A consideration for the effectiveness of the SBPS is the frequency recovery stage fast fourier-transform (FFT)-bin spacing. This FFT bin spacing determines the maximum residual carrier frequency offset (CFO) that remains after course frequency recovery (equation 2.3).

This residual CFO causes a phase drift across one SBPS processing block of B symbols:

$$\Delta\phi_{block} \approx 2\pi\Delta f_e B T_s$$

Since $T_s = 1/F_s$, substituting the worst-case residual CFO gives:

$$\Delta\phi_{block,max} = 2\pi\left(\frac{F_s}{2N_{FFT}}\right)B\left(\frac{1}{F_s}\right)$$

which simplifies to:

$$\Delta\phi_{block,max} = \pi \frac{B}{N_{FFT}}$$

So, the worst-case accumulated phase drift over a block depends only on block size and FFT, not directly on sampling frequency.

For the proposed SBPS with a $\pi/4$ unwrapping threshold, the following condition needs to be specified:

$$\Delta\phi_{block,max} \leq \pi/4$$

which gives

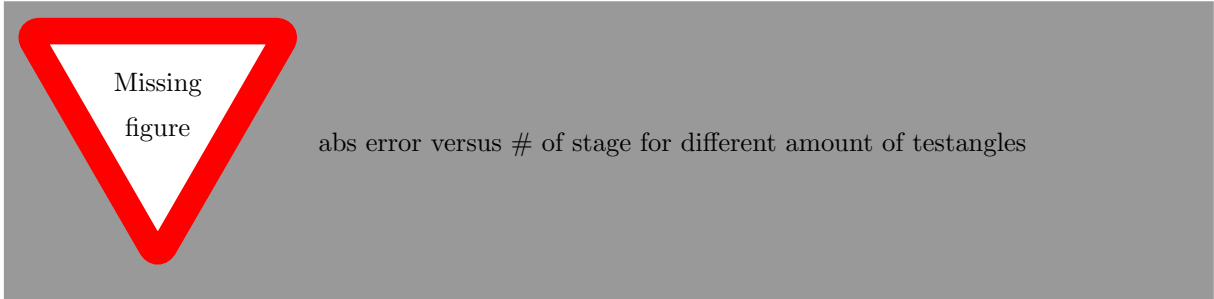
$$N_{FFT} > 4B \quad (3.2)$$

For example, a blocksize of $B = 64$ would need at least $N_{FFT} > 256$ for the SBPS to be effective without introducing (additional) cycle slips. This is a theoretical minimum, and in practice a margin is recommended because of the presence of linewidth-induced phase noise and estimation errors.

find ref for
binsize related
to accuracy?

Further, the block size has a dual effect on estimation quality. On the one hand, using more symbols in the metric accumulation improves robustness against additive noise, because the random perturbations introduced by noise and symbol decisions are averaged over a larger number of samples [22]. On the other hand, a larger block spans a longer time interval, so the phase variation caused by residual frequency offset and phase noise becomes larger within the block. As a result, a single phase estimate becomes less representative for all symbols in the block.

This trade-off is one of the main motivations for the proposed two-part architecture: SBPS provides an accurate estimate of the middle-block phase, while the spread estimation block reconstructs the phase evolution around that midpoint.



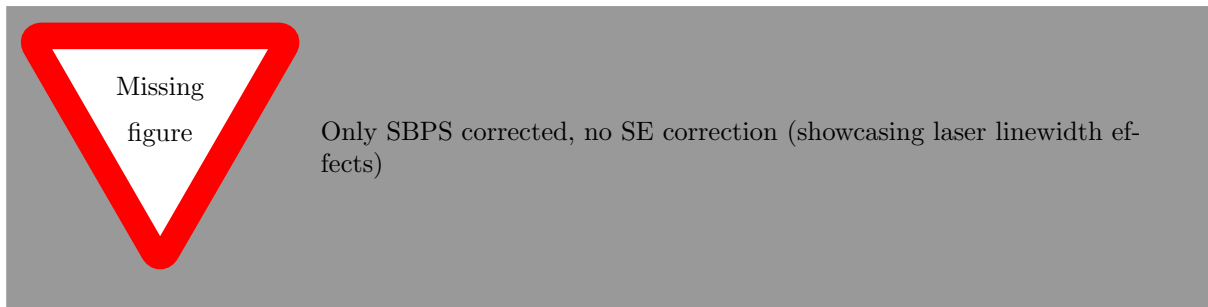
maybe add
figure with
complexity
versus abs
error?

3.3 Spread estimation algorithm

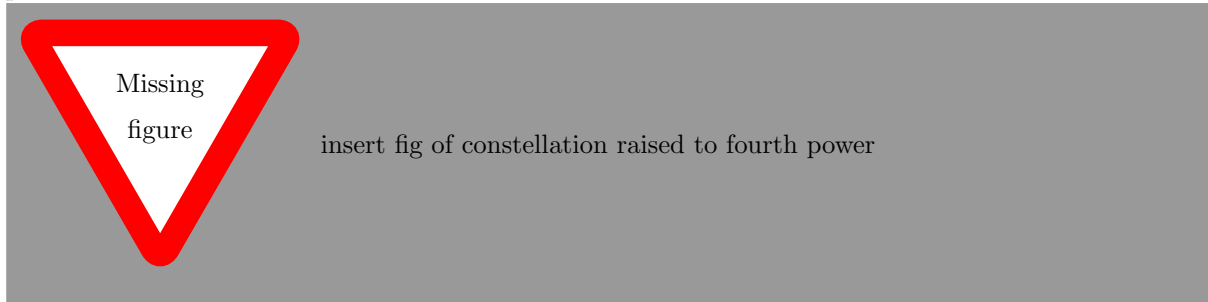
Since SBPS only generates one phase estimation per block, and since the practical effects of laser linewidth drift are still present, only correcting the block with a single SBPS phase estimation would still not resolve this problem as may be seen in figure

explain the
SE first, then
the criteria
for usable
symbols, then
input logic

REF



. A parallel calculation will estimate this residual phase spread across the block. This is done by raising the symbols to the fourth power, thereby removing the modulation and only keeping the true phase information.



3.3.1 Input symbol consideration

3.4 Phase unwrapping

3.5 Phase compensation

3.6 Expected performance

Hardware implementation

Here you show: VHDL design bit widths latencies signals testbenches block diagrams

4.1 System overview

4.2 Data representation and scaling

explain why samples are in I/Q fxp, and testangles are cos/sin fxp (more difficult to calculate angles with, easier to apply angles to samples)

4.3 Serialized blind phase search block

The hardware implementation follows a staged refinement architecture, shown in Fig. 4.1. The same received block

$$\bar{r} = (r_0, r_1, \dots, r_{W-1})$$

is presented to every stage, while only the accumulated phase estimate is updated from stage to stage. At stage m , two candidate phase updates are evaluated around the previously selected estimate ϕ_{m-1} , namely $\phi_{m-1} - \phi_t$ and $\phi_{m-1} + \phi_t$. The candidate that produces the smaller block error is selected as the stage output ϕ_m , which is then forwarded to the next stage. By cascading several stages with progressively smaller test angles, the architecture converges towards the phase estimate associated with the centre of the block.

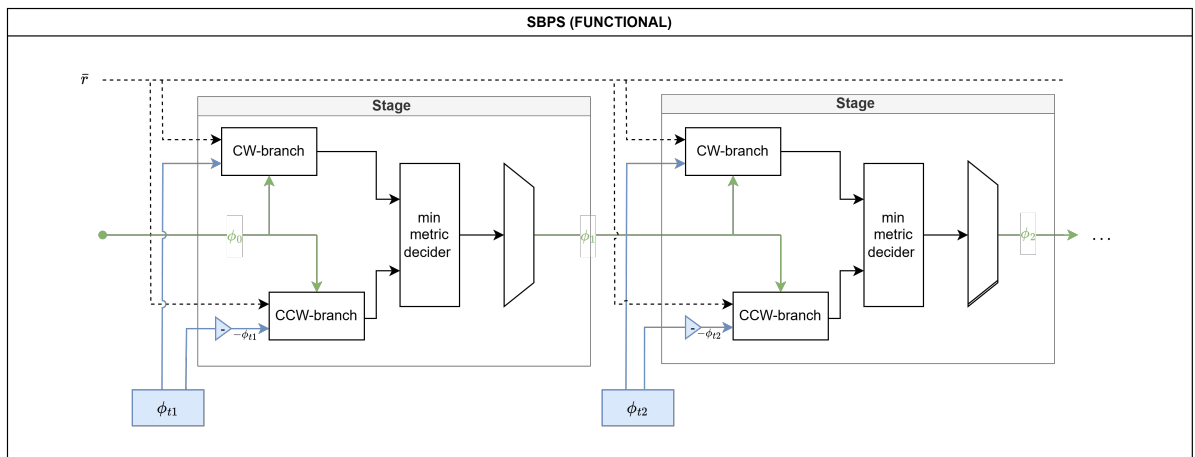


Figure 4.1: Functional diagram of the full SBPS architecture

4.3.1 Functional operation

Figure 4.2 shows the internal organisation of a single stage. The upper branch corresponds to the clockwise (CW) update and evaluates $\phi_{m-1} - \phi_t$, whereas the lower branch corresponds to the counter-clockwise (CCW) update and evaluates $\phi_{m-1} + \phi_t$. Each branch contains a *rotate metric slice* unit that processes the complete input block using its own candidate phase. The resulting block metrics are compared by the *min metric decider*, and the associated candidate phase is selected by the final multiplexer (MUX). In this way, each stage performs a binary phase refinement around the current estimate.

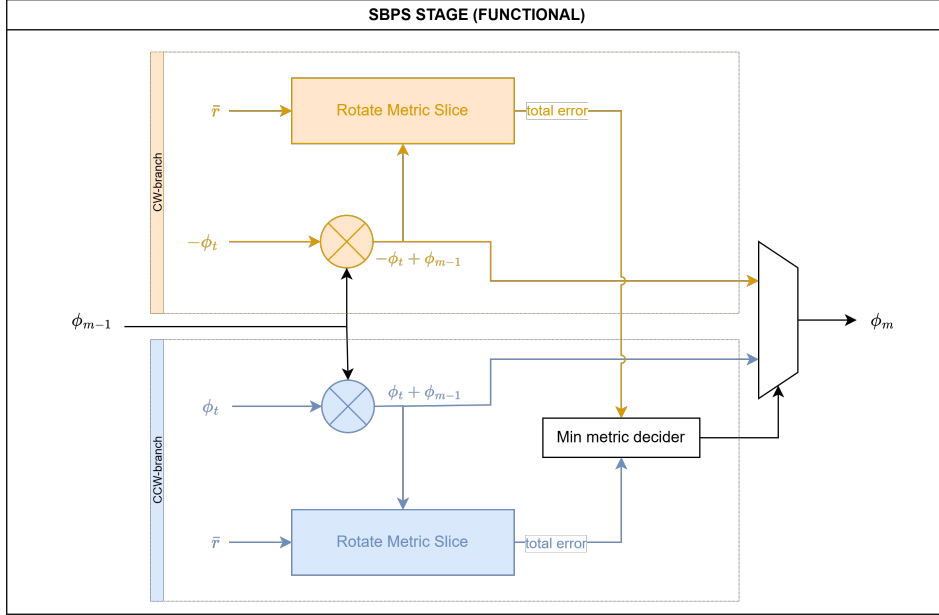


Figure 4.2: Functional diagram of the SBPS stage

The *rotate metric slice* unit is the core computational block of the design and is detailed in Fig. 4.3. For every symbol in the block, the candidate phase is used to rotate the received sample, the rotated sample is sliced to the nearest constellation point, and the squared Euclidean distance between the rotated and sliced values is computed. These per-symbol errors are then accumulated over the full block window to obtain a single decision metric for the branch. All symbols in the block are therefore processed in parallel inside each branch, while the serial refinement occurs across the SBPS stages.

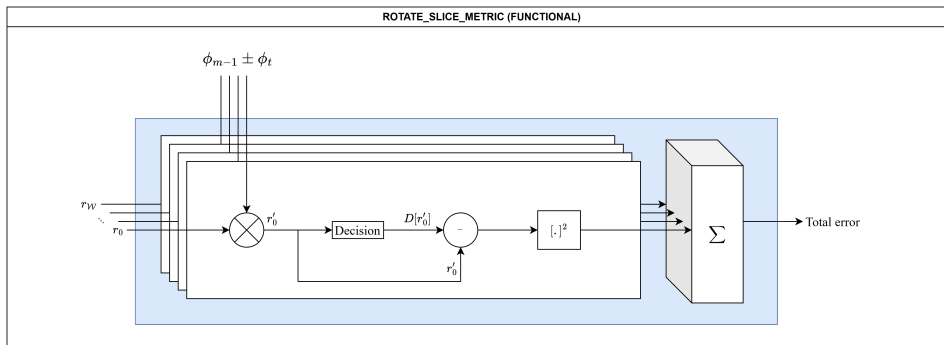


Figure 4.3: Functional diagram of the *rotate metric slice* unit

4.3.2 Data flow

The data-oriented view of the implementation is shown in Figs. 4.4, 4.5, and 4.6.

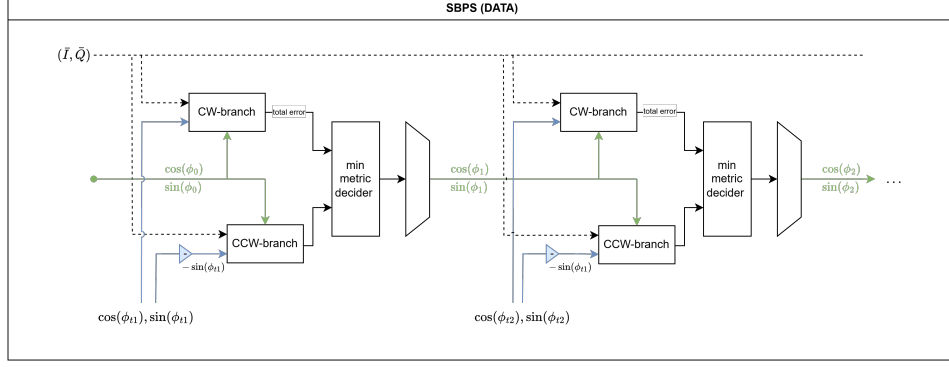


Figure 4.4: Datapath diagram of the full SBPS architecture

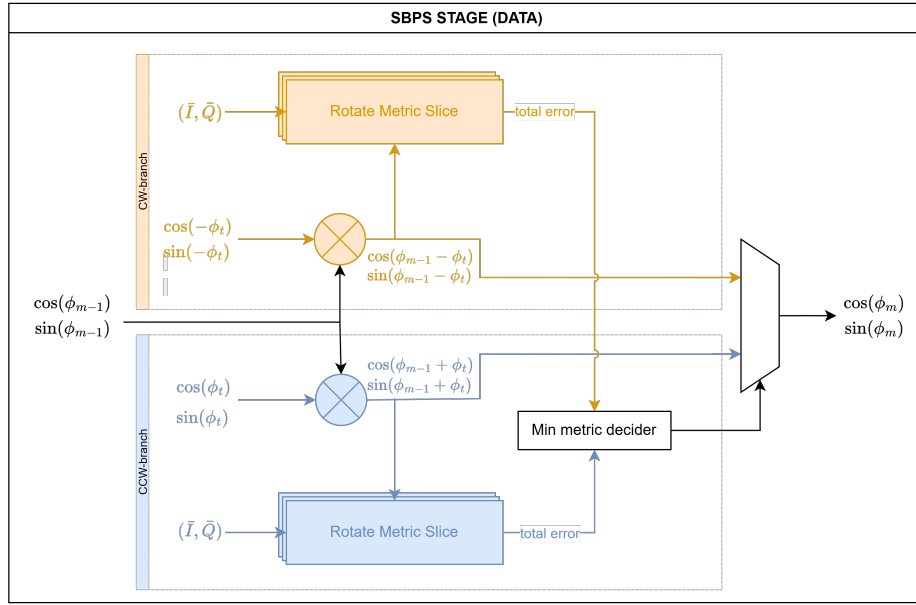


Figure 4.5: Datapath diagram of the SBPS stage

In the implemented datapath, phase is not represented as an explicit angle word. Instead, each phase value is carried by its trigonometric components $(\cos(\phi), \sin(\phi))$ in fixed-point format. This allows the candidate phases to be generated directly by the *angle add* block from the previous estimate $(\cos(\phi_{m-1}), \sin(\phi_{m-1}))$ and the stage test angle $(\cos(\phi_t), \sin(\phi_t))$:

$$\cos(\phi_{m-1} \pm \phi_t) = \cos(\phi_{m-1}) \cos(\phi_t) \mp \sin(\phi_{m-1}) \sin(\phi_t), \quad (4.1)$$

$$\sin(\phi_{m-1} \pm \phi_t) = \sin(\phi_{m-1}) \cos(\phi_t) \pm \cos(\phi_{m-1}) \sin(\phi_t). \quad (4.2)$$

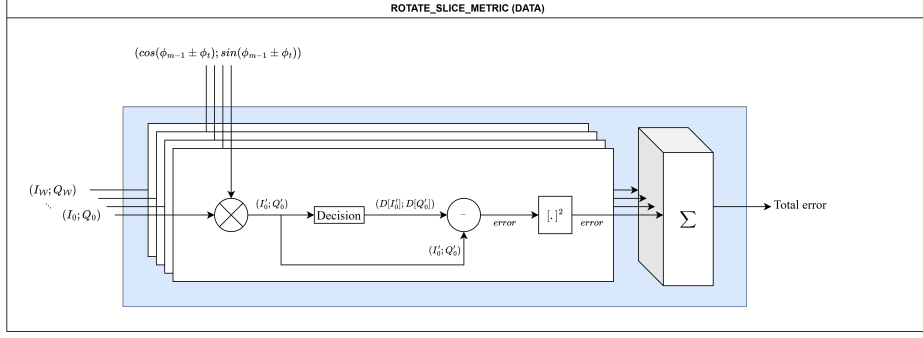
Once a candidate phase has been formed, the received samples are rotated in the I/Q plane. For a candidate angle

$$\theta \in \{\phi_{m-1} - \phi_t, \phi_{m-1} + \phi_t\},$$

the rotator computes

$$\begin{aligned} I'_k &= I_k \cos(\theta) + Q_k \sin(\theta), \\ Q'_k &= -I_k \sin(\theta) + Q_k \cos(\theta), \end{aligned} \quad (4.3)$$

for every symbol $r_k = I_k + jQ_k$ in the block. The rotated symbol is then denoted by $r'_k = I'_k + jQ'_k$.

Figure 4.6: Datapath diagram of the *rotate metric slice* unit

For standard 16QAM operation, the slicer maps each rotated sample to the nearest reference point $D[r'_k]$. The axis decisions are derived from the conventional inner-outer thresholds, with $\pm 2s$ defining the transition between the inner and outer levels of the constellation. The resulting single-symbol error is

$$e_k = |r'_k - D[r'_k]|^2 = (I'_k - D_I[r'_k])^2 + (Q'_k - D_Q[r'_k])^2, \quad (4.4)$$

and the branch metric is obtained by accumulation over the complete block:

$$E = \sum_{k=0}^{\mathcal{W}-1} e_k. \quad (4.5)$$

This accumulated metric is the quantity used by the *min metric decoder* to select the surviving branch.

4.3.3 Timing flow

The timing behaviour of the full chain, of one SBPS stage, and of the *rotate metric slice* unit is summarised in Figs. 4.7, 4.8, and 4.9, respectively. The control path uses single-cycle valid pulses rather than a ready/valid handshake, so temporal alignment is achieved entirely through fixed pipeline delays.

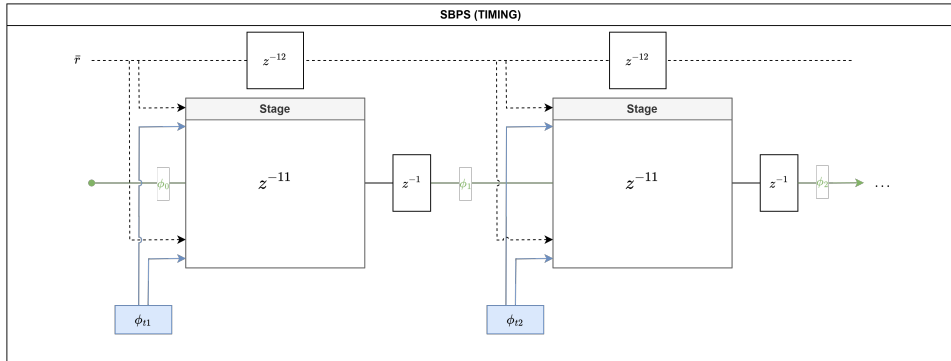


Figure 4.7: Timing diagram of the full SBPS algorithm

At the stage input, the two candidate phases are first generated while the input sample block is delayed in parallel so that both reach the branch datapaths in synchrony. The *angle add* operation introduces three elapsed clock cycles, and the sample block is delayed by the same amount before it enters the *rotate metric slice* unit.

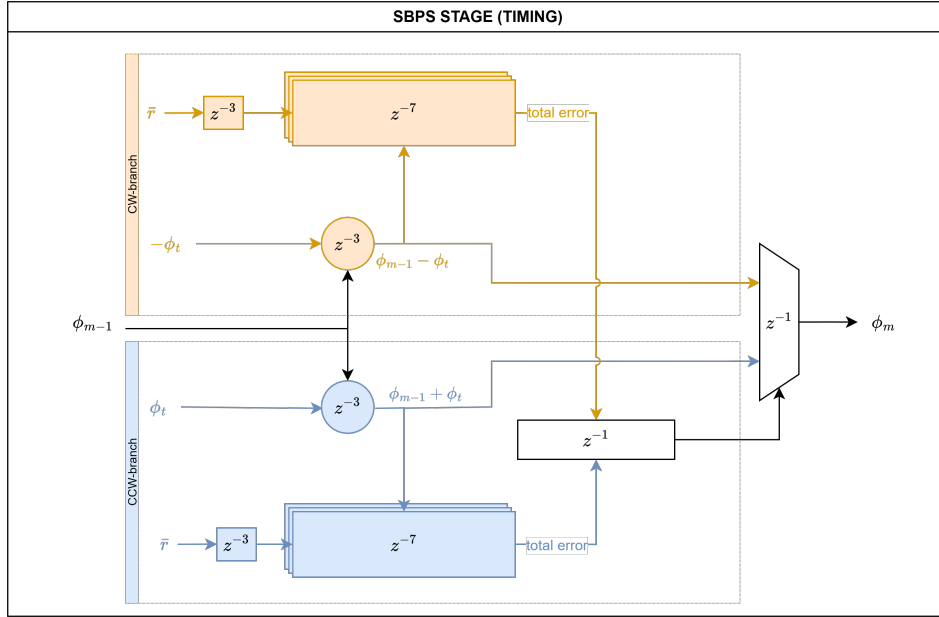
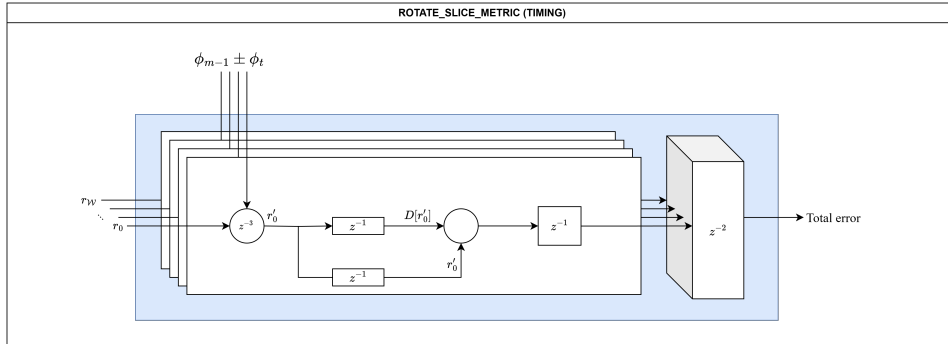


Figure 4.8: Timing diagram of the SBPS stage

Inside the *rotate metric slice* unit, the rotator contributes three cycles, the slicer and sample-alignment logic contribute one cycle, the single-symbol metric calculation contributes one cycle, and the summation plus output registration contribute two further cycles. The branch metric is therefore available seven cycles after entry into the *rotate metric slice* unit, or ten elapsed cycles after the stage launch when the initial angle-generation and input-alignment delay are included.

Figure 4.9: Timing diagram of the *rotate metric slice* unit

After both branch metrics have become available, the *min metric decider* requires one clock cycle to determine the winning branch, and the final stage output is registered one cycle later. Consequently, the phase-selection datapath of a stage spans eleven clock cycles, while the registered stage-valid output is asserted after twelve clock cycles. When latency is reported at the boundary of the complete SBPS chain rather than at a single stage boundary, one additional launch cycle must be included because the top-level chain registers the hand-off into the first stage.

4.3.4 16QAM-specific adaptations

For 16QAM, the slicer supports both conventional axis-based decisions and an optional radius-directed mode in the early refinement stages. In the conventional case, the decision on each axis is determined by the inner-outer threshold at $\pm 2s$. In the radius-directed variant, an additional threshold l is introduced to identify samples that should be mapped directly to the outer ring of the constellation. This is beneficial in the first stages of the search, where the residual phase uncertainty is still relatively large and a purely axis-based decision can be less reliable. Once the phase estimate has been sufficiently refined, the later

stages revert to the standard 16QAM slicer, so that the final metric evaluation follows the nominal decision regions.

not well
enough ex-
plained

4.4 Spread estimation block

The hardware implementation is split up in the different operational sections of the algorithm, namely the input selector (1), fourth-power demodulators (2), the spread estimator (3), the sign estimator (4) and finally the filters (5). The layout of these modules and their relative position is illustrated on Fig 4.10. Each section will individually discuss the modules in terms of its functional operation, dataflow and timing flow in order to get a clearer picture of the RTL implementation.

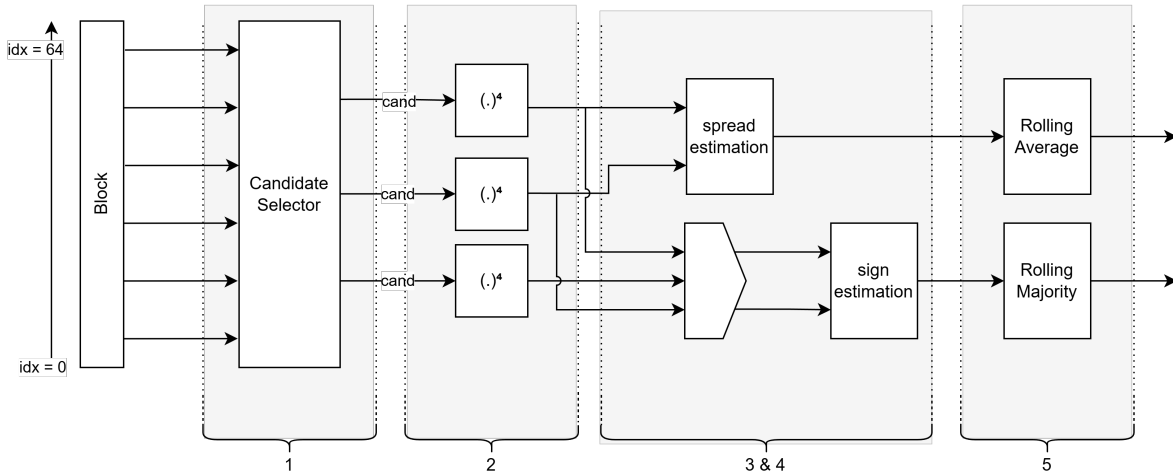


Figure 4.10: Top lay-out of SE block

4.4.1 Input selector

Functional operation

As introduced in Section 3.3.1, the spread and direction estimation blocks require a set of representative symbols from the input block. However, not all symbols are suitable for this purpose. Therefore, an input selection stage is required to extract valid candidate symbols from different regions of the block.

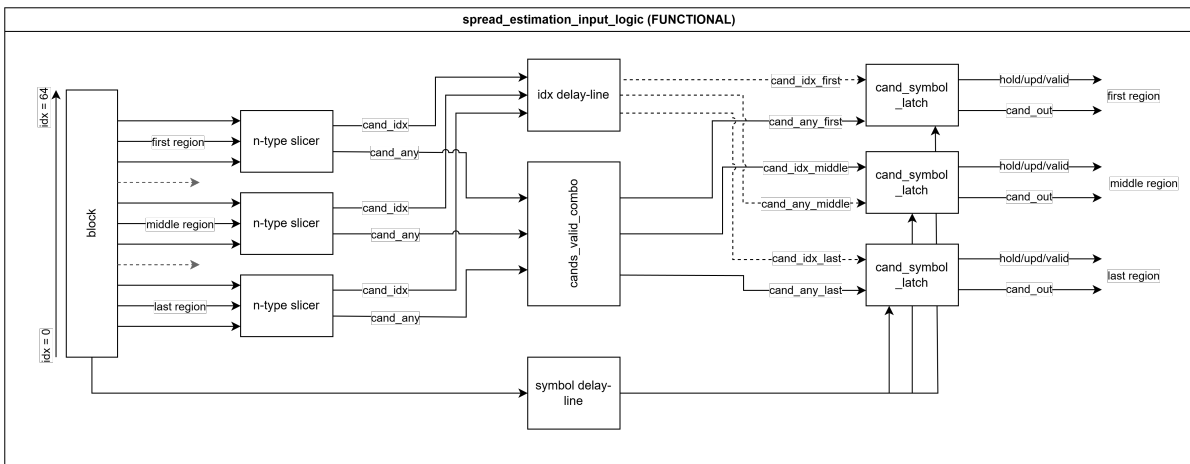


Figure 4.11: Functional diagram of the SE input selector

The input selector operates on the full block of received symbols $\bar{r}$, and outputs three candidate symbols x , y , and z , corresponding to the first, middle, and last regions of the block, respectively. These candidates are subsequently used for spread and direction estimation.

The selection process consists of three sequential steps:

1. Identification of valid candidate symbols within each region of the block.
2. Validation of the combination of candidates across regions.
3. Latching or updating of the selected symbols for further processing.

These steps are implemented by the *n-type slicer*, *cands_valid_combo*, and *cand_symbol_latch* modules, respectively.

Each region (first, middle, last) is processed by an independent *n-type slicer*. This module evaluates a small subset of symbols (three in this implementation) and determines whether a suitable candidate is present. A symbol is considered valid if it lies on the inner or outer ring of the constellation, which ensures reliable spread estimation (this criterion is further elaborated in the algorithmic description of the spread estimator). The index of the first valid symbol is returned via the *cand_idx* bus. If no valid candidate is found, the *cands_any* flag is deasserted.

The *cands_valid_combo* module evaluates the availability of candidates across the three regions. A valid combination is only accepted if at least two regions provide a candidate symbol. If this condition is not met, all candidate flags are suppressed. This constraint reduces the likelihood of unreliable spread or direction estimates, which would otherwise arise from insufficient spatial information within the block.

Finally, the *cand_symbol_latch* module selects and outputs the candidate symbols. For each region where the *cands_any* flag is asserted, the corresponding symbol is extracted from the input block using the provided *cand_idx*. If valid candidates are present, the outputs are updated and the *update* signal is asserted. Otherwise, the previous outputs are retained and the *hold* signal is asserted. In this way, consistency is maintained when no valid candidate combination is available.

Timing flow

The timing behaviour of the input selector is illustrated in Figure 4.12.

The *n-type slicer* requires two clock cycles to determine and register a valid candidate index. The subsequent *cands_valid_combo* stage introduces an additional cycle to produce stable validity signals.

To ensure correct operation of the *cand_symbol_latch*, all inputs must be temporally aligned. In particular, the candidate indices and their corresponding validity flags must be synchronized with the original input symbols. To achieve this, delay elements are inserted in the data path. The input symbol block $\bar{r}$ is delayed by three clock cycles, matching the combined latency of the slicer and combination stages. The *cand_idx* signals are delayed by one cycle to align with the output of the *cands_valid_combo* module.

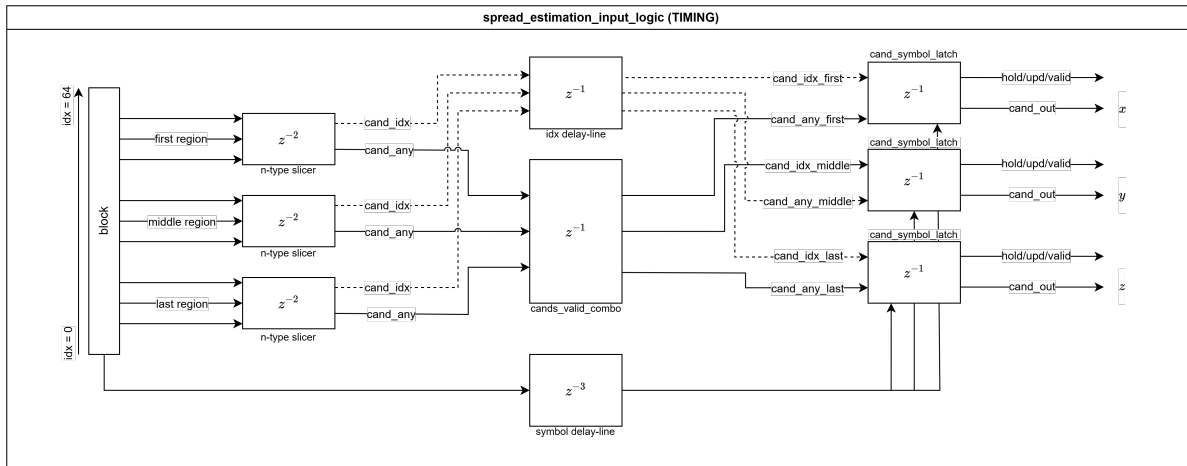


Figure 4.12: Timing flow diagram of the SE input selector

The *cand_symbol_latch* itself introduces one additional clock cycle to register the selected symbols. As a result, the total latency of the input selector amounts to four clock cycles.

4.4.2 Fourth-power I/Q calculators

Functional operation

The fourth-power calculators are an important part of the system. The type of calculations needed to perform a fourth-power on I/Q samples require multiplications, more of them than any other type of calculation that needs to be done for the low-complexity carrier recovery system.

Data flow

Timing flow

4.4.3 Spread estimation

Functional operation

1. compute dotproduct
2. compute denominator (inner + outer flags go to radius LUT with as output inversed reciprocal multiplication factors)
3. multiplication of dotproduct with rec. mult. factor to produce $\cos(|4\theta|)$.
4. arccos LUT

important,
explain well!

Data flow

Timing flow

4.4.4 Sign estimation

Functional operation

sign pair decider go to first + middle or middle + last combination with that combo complex cross sign

Data flow

Timing flow

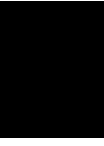
4.4.5 Filters

Timing flow

4.5 Phase correction block

4.6 Top-level integration

This page is intentionally left blank.



Verification and simulation

You already have: python vectors testbenches error plots Put all here.

5.1 Simulation model

5.2 Test vector generation

5.3 VHDL testbench setup

5.4 Error analysis

5.4.1 Serialized blind phase search

5.4.2 Phase spread estimation

The absolute phase error between the floating-point model and the fixed-point implementation remains below ... rad for all tested blocks.

5.4.3 Phase unwrapper

5.4.4 Phase compensator

This page is intentionally left blank.

Possible results:

- phase error
- EVM
- resource usage
- timing
- block size effect

6.1 Serial blind phase search results



SNR versus BER per number of stages (so constant CFO and dnu)

6.2 Phase spread estimation results

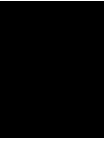
6.3 Error versus SNR

6.4 Resource usage

6.5 Latency

6.6 Comparison with reference method

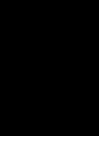
This page is intentionally left blank.



Discussion

- 7.1 Limitations
- 7.2 Possible improvements
- 7.3 Design trade-offs

This page is intentionally left blank.



Conclusion

- 8.1 Summary
- 8.2 Achieved objectives
- 8.3 Future work

This page is intentionally left blank.

Bibliography

- [1] Coherent Corp., “Coherent to demonstrate next-generation transceiver and semiconductor laser technology for 800g and 1.6t transmission in ai networks at ecoc 2023.” <https://www.coherent.com/news/press-releases/coherent-to-demo-next-gen-transceiver-and-semiconductor-tech-at-ecoc>, Oct. 2023. accessed April 9, 2026.
- [2] Jingchi Cheng, et al., “Comparison of coherent and imdd transceivers for intra datacenter optical interconnects.” https://www.researchgate.net/publication/331337485_Comparison_of_Coherent_and_IMDD_Transceivers_for_Intra_Datacenter_Optical_Interconnects, Jan. 2019. accessed April 9, 2026.
- [3] M. S. Wartak, *Computational Photonics: An Introduction with MATLAB*. Cambridge University Press, 2013. Chapter 10: Optical receivers.
- [4] J. M. Kahn and K.-P. Ho, “Spectral efficiency limits and modulation/detection techniques for dwdm systems,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 10, no. 2, pp. 259–272, 2004.
- [5] S. J. Savory, “Digital coherent optical receivers: Algorithms and subsystems,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 16, no. 5, pp. 1164–1179, 2010.
- [6] M. Kuschnerov, F. N. Hauske, K. Piyawanno, B. Spinnler, M. S. Alfiad, A. Napoli, and B. Lankl, “Dsp for coherent single-carrier receivers,” *Journal of Lightwave Technology*, vol. 27, no. 16, pp. 3614–3622, 2009.
- [7] X. Zhou, R. Urata, and H. Liu, “Beyond 1 tb/s intra-data center interconnect technology: Im-dd or coherent?,” *Journal of Lightwave Technology*, 2020.
- [8] K. Kikuchi, “Fundamentals of coherent optical fiber communications,” *Journal of Lightwave Technology*, 2016.
- [9] E. Ip and J. M. Kahn, “Feedforward carrier recovery for coherent optical communications,” *Journal of Lightwave Technology*, vol. 25, no. 9, pp. 2675–2692, 2007.
- [10] T. Pfau, S. Hoffmann, and R. Noé, “Hardware-efficient coherent digital receiver concept with feed-forward carrier recovery for m-qam constellations,” *Journal of Lightwave Technology*, vol. 27, no. 8, pp. 989–999, 2009.
- [11] I. Roudas, “Coherent optical communication systems,” in *Optical Networks: WDM Systems and Networks*, Springer, 2011.
- [12] A. Leven, N. Kaneda, U.-V. Koc, and Y.-K. Chen, “Frequency estimation in intradyne reception,” *IEEE Photonics Technology Letters*, vol. 19, no. 6, pp. 366–368, 2007.
- [13] M. Selmi, Y. Jaouen, and P. Ciblat, “Accurate digital frequency offset estimator for coherent polmux qam transmission systems,” in *Proceedings of the European Conference on Optical Communication (ECOC)*, 2009. Paper P3.08, Vienna.
- [14] J. C. Rasmussen, “Advances in coherent detection algorithms,” in *Proceedings of SPIE*, vol. 7960, 2011.
- [15] A. Devices, “Mixed-signal and dsp design techniques: The dft/fft (bin spacing and frequency resolution).” PDF technical handbook chapter.
- [16] H.-P. . A. Technologies, “Spectrum and signal analyzer measurements and noise.” Technical measurement guide (FFT/bin-based resolution and estimation effects).

- [17] T. Yang *et al.*, “Linewidth-tolerant and multi-format carrier phase estimation based on blind phase search,” *Optics Express*, 2018.
- [18] J. E. Volder, “The cordic trigonometric computing technique,” *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [19] P. K. Meher, J. Valls, T.-B. Juang, K. Sridharan, and K. Maharatna, “50 years of cordic: Algorithms, architectures, and applications,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 56, no. 9, pp. 1893–1907, 2009.
- [20] A. (Xilinx), “Cordic v6.0 logiccore ip product guide (pg105),” 2021. Release date 2021-08-06.
- [21] e. a. Jonas De Busscher, Axl Bomhals, “Low complexity carrier recovery system design for baudrate sampled coherent datacenter interconnects,” *IEEE Communications Letters*, 2025.
- [22] D. A. A. Mello, F. A. Barbosa, and J. D. Reis, “Interplay of probabilistic shaping and the blind phase search algorithm,” *Journal of Lightwave Technology*, vol. 36, p. 5096–5105, Nov. 2018.

This page is intentionally left blank.

This page is intentionally left blank.

